

Netzwerksicherheit

Protokolle

ISO/OSI Modell

Das ISO/OSI-Modell (International Organization for Standardization/Open Systems Interconnection) ist ein Referenzmodell, das zur Beschreibung von Netzwerkprotokollen und deren Kommunikation dient. Es besteht aus sieben Schichten, die verschiedene Funktionen innerhalb einer Netzwerkkommunikation übernehmen. Die sieben Schichten sind:

1. Physikalische Schicht (Physical Layer): Zuständig für die Übertragung von rohen Bitströmen über ein physisches Medium, wie Kabel oder Funkwellen. Sie definiert Spannungspegel, Timing, Datenraten und physische Verbindungen.
2. Datenverbindungsschicht (Data Link Layer): Verantwortlich für die sichere und fehlerfreie Übertragung von Datenpaketen zwischen benachbarten Netzwerkknoten. Hier werden Fehlererkennung und Fehlerkorrektur, sowie Flusskontrolle und Rahmenbildung (Framing) behandelt.
3. Netzwerkschicht (Network Layer): Hier wird die Routing-Funktionalität bereitgestellt, um Datenpakete von einem Netzwerkknoten zum anderen über unterschiedliche Netzwerke zu übertragen. Sie ist auch verantwortlich für die Adressierung von Netzwerkknoten und die Aufteilung von Datenpaketen in kleinere Segmente, falls erforderlich. Im Internet finden sich in dieser Schicht typischerweise Protokolle wie IP und ICMP.
4. Transportschicht (Transport Layer): Die Transportschicht stellt eine Ende-zu-Ende-Kommunikation zwischen Quelle und Ziel bereit, indem sie Datenpakete in Segmente aufteilt, diese sequenziert und bei Bedarf neu überträgt. Sie ist auch verantwortlich für die Flusskontrolle und die Bereitstellung von verbindungsorientierten oder verbindungslosen Kommunikationsdiensten. Auf dieser Ebene sind auch die im Internet verwendeten Protokolle TCP und UDP einzuordnen.
5. Sitzungsschicht (Session Layer): Die Sitzungsschicht ermöglicht den Aufbau, die Verwaltung und den Abbau von Verbindungen zwischen Anwendungen auf verschiedenen Netzwerkknoten. Sie verwaltet auch die Synchronisation und Wiederherstellung von unterbrochenen Kommunikationssitzungen.
6. Präsentationsschicht (Presentation Layer): Diese Schicht ist verantwortlich für die Übersetzung, Verschlüsselung und Kompression von Daten, um sie für die Anwendungsschicht verständlich zu machen. Sie stellt sicher, dass die Daten in einem formatierten und für die Anwendung geeigneten Format übertragen werden.
7. Anwendungsschicht (Application Layer): Die oberste Schicht des OSI-Modells stellt Schnittstellen für Anwendungen bereit, um Netzwerkdienste

nutzen zu können. Sie definiert Protokolle für die Kommunikation zwischen Anwendungen, wie E-Mail, Dateiübertragung und Web-Browsing.

Anmerkung: Einige in der Praxis weit verbreiteten Protokolle wie TLS (Transport Layer Security) lassen sich im ISO/OSI Modell nicht eindeutig einordnen. TLS wäre zum Beispiel in den Layern 5 und 6 (Session und Presentation) anzusiedeln.

Das Prinzip der **Kapselung** bezieht sich auf den Prozess, bei dem jede Schicht des ISO/OSI-Modells Daten von der darüberliegenden Schicht empfängt, diese mit zusätzlichen Informationen wie zum Beispiel Header-Informationen versieht und an die darunterliegende Schicht weitergibt. Jede Schicht behandelt nur die für sie relevanten Informationen und ist sich der anderen Schichten nicht bewusst. Diese Kapselung trägt zur Abstraktion bei und ermöglicht eine klar definierte Trennung der Verantwortlichkeiten zwischen den verschiedenen Schichten des Modells.

Angriffsarten

Sniffing

Sniffing-Angriffe beziehen sich auf das unerlaubte Abhören, Erfassen und Analysieren von Datenpaketen, die in einem Netzwerk übertragen werden. Bei einem Sniffing-Angriff nutzt ein Angreifer eine Software oder Hardware, die als "Sniffer" bezeichnet wird, um den Datenverkehr in einem Netzwerk zu überwachen und Informationen daraus abzuleiten. Solche Angriffe können sowohl in kabelgebundenen als auch in drahtlosen Netzwerken stattfinden.

Um sich vor Sniffing-Angriffen zu schützen, können verschiedene Maßnahmen ergriffen werden:

1. Verschlüsselung: Die Verwendung von Verschlüsselungsprotokollen wie TLS (Transport Layer Security) oder VPNs (Virtual Private Networks) kann dazu beitragen, den Datenverkehr vor Sniffen zu schützen, indem sie die Kommunikation verschlüsseln und für den Angreifer unlesbar machen.
2. Zugriffskontrolle: Durch die Implementierung von Zugriffskontrollen auf Netzwerkebene kann der Zugang zum Netz auf bestimmte Geräte beschränkt werden. Diese können zum Beispiel anhand der MAC-Adresse oder Zertifikaten erfolgen.
3. Segmentierung des Netzwerks: Die Aufteilung des Netzwerks in separate Segmente, die voneinander isoliert sind, kann dazu beitragen, den Umfang eines Sniffing-Angriffs einzuschränken und die Ausbreitung des Angriffs zu verhindern.
4. Überwachung und Erkennung: Die Implementierung von Intrusion-Detection-Systemen (IDS) und regelmäßige Überwachung des Netzwerkverkehrs können dazu beitragen, verdächtige Aktivitäten zu erkennen und schnell auf potenzielle Sniffing-Angriffe zu reagieren.

Spoofing

Spoofing-Angriffe sind eine Art von Cyberangriff, bei dem ein Angreifer seine Identität oder die Identität einer vertrauenswürdigen Person oder eines Geräts vortäuscht, um Zugang zu sensiblen Informationen zu erhalten, die Kommunikation zu manipulieren oder Systeme und Netzwerke zu kompromittieren. Bei Spoofing-Angriffen gibt es verschiedene Techniken, die sich auf unterschiedliche Aspekte von Netzwerken und Kommunikationssystemen beziehen. Hier sind einige gängige Spoofing-Angriffsmethoden:

1. **IP-Spoofing:** Bei diesem Angriff gibt der Angreifer eine gefälschte IP-Adresse an, die der eines vertrauenswürdigen Netzwerkgeräts ähnelt, um den Datenverkehr zu manipulieren oder unbemerkt Zugang zu einem geschützten Netzwerk zu erhalten. IP-Spoofing kann auch als Teil von Denial-of-Service-Angriffen verwendet werden, um die tatsächliche Quelle des Angriffs zu verschleiern.
2. **ARP-Spoofing (Address Resolution Protocol Spoofing):** Bei dieser Methode täuscht der Angreifer die ARP-Tabellen in einem lokalen Netzwerk, indem er gefälschte ARP-Nachrichten sendet. Dadurch kann der Angreifer den Datenverkehr zwischen Netzwerkgeräten abfangen, manipulieren oder blockieren.
3. **E-Mail-Spoofing:** Bei diesem Angriff fälscht der Angreifer die Absenderadresse einer E-Mail, um den Empfänger glauben zu lassen, dass die Nachricht von einer vertrauenswürdigen Quelle stammt. Dies kann verwendet werden, um vertrauliche Informationen zu erlangen, Phishing-Angriffe durchzuführen oder Malware zu verbreiten.
4. **DNS-Spoofing (Domain Name System Spoofing):** Bei dieser Technik schickt der Angreifer gefälschte Antworten auf DNS-Anfragen an ein Opfer und kann damit zum Beispiel Datenverkehr auf bösartige oder gefälschte Websites umleiten. Dies kann dazu führen, dass Benutzer ihre Anmeldedaten oder andere vertrauliche Informationen preisgeben, ohne zu wissen, dass sie auf einer gefälschten Website sind. Der Begriff DNS-Poisoning bezeichnet hingegen Angriffe, bei denen auf verschiedene Art und Weise ein DNS-Server selbst so manipuliert wird, dass er falsche Antworten liefert.

Flooding

Switches besitzen typischerweise Tabellen, in denen sie speichern, welche Geräte (anhand deren MAC-Adresse) mit welchem Anschluss (Port) verbunden sind. Pakete werden dann direkt nur an das Zielgerät zugestellt, was Sniffing-Angriffe verhindert. Bei sogenanntem MAC-Flooding schickt ein Angreifer massenweise Pakete mit gefälschten MAC-Adressen an einen Switch, bis dessen interner Speicher voll mit gefälschten Adressen ist. Viele Geräte schalten in so einem Fall in einen Fallback-Modus, und verhalten sich wie ein Hub. Werden danach neue Geräte an den Switch angesteckt, so werden Pakete an diese an alle Ports

weitergeleitet und sind anfällig auf Sniffing-Angriffe. Ein solcher Angriff kann zum Beispiel unter Linux mit dem Kommando `macof` durchgeführt werden.

Praxisbeispiele

iptables

Iptables ist eine unter Linux/Unix-Systemen weit verbreitete Software zur Verwaltung von IP-Paketfilterregeln in Linux-basierten Systemen. Es ermöglicht das Erstellen, Ändern und Anwenden von Regeln, die den Datenverkehr auf der Grundlage von IP-Adressen, Ports, Protokollen und anderen Kriterien steuern. Iptables dient als Schnittstelle zur Netfilter-Engine im Linux-Kernel und bietet eine hohe Flexibilität bei der Konfiguration von Firewall-Regeln. Anderen Firewalls wie `ufw` unter Ubuntu setzen auf iptables auf.

Der Funktionsumfang von Iptables umfasst:

1. Paketfilterung: Filterung von eingehendem, ausgehendem und durchlaufendem Netzwerkverkehr.
2. Netzwerkadressübersetzung (NAT): Ändern der Quell- oder Ziel-IP-Adresse von Paketen.
3. Portweiterleitung: Umleiten von Paketen von einem Port zu einem anderen.
4. Verbindungsnachverfolgung: Protokollierung und Analyse des Netzwerkverkehrs.

Bei Iptables sind Chains (Ketten) grundlegende Strukturen, die zur Organisation von Firewall-Regeln verwendet werden. Jede Kette besteht aus einer Reihe von Regeln, die der Reihenfolge nach angewendet werden. Iptables verarbeitet Pakete, indem es sie durch verschiedene Chains leitet und dabei die jeweiligen Regeln auswertet, um zu entscheiden, welche Aktion (Ziel) für jedes Paket ausgeführt werden soll (z. B. ACCEPT, DROP, REJECT).

Es gibt drei vordefinierte Chains in Iptables:

1. INPUT: Diese Kette verarbeitet alle eingehenden Pakete, die an das System selbst gerichtet sind. Das bedeutet, dass Pakete, die für das System bestimmt sind, durch die INPUT-Kette geleitet und dort anhand der konfigurierten Regeln ausgewertet werden.
2. OUTPUT: Diese Kette betrifft alle ausgehenden Pakete, die vom System selbst initiiert werden. Das bedeutet, dass Pakete, die vom System gesendet werden, durch die OUTPUT-Kette geleitet und dort anhand der konfigurierten Regeln ausgewertet werden.
3. FORWARD: Die FORWARD-Kette ist für Pakete zuständig, die weder an das System selbst gerichtet noch von ihm initiiert sind, sondern lediglich durch das System hindurchgeleitet werden. Dies ist der Fall, wenn das System als Router oder Gateway fungiert. In der FORWARD-Kette werden Pakete anhand der konfigurierten Regeln ausgewertet.

Jede Kette hat eine Standardrichtlinie (Policy), die angibt, was mit einem Paket geschehen soll, wenn keine der Regeln in der Kette zutrifft. Die Standardrichtlinie kann auf ACCEPT, DROP oder REJECT gesetzt werden.

Neben den vordefinierten Chains können Benutzer auch eigene benutzerdefinierte Chains erstellen. Diese benutzerdefinierten Chains können verwendet werden, um spezielle Regelsets zu organisieren und die Firewall-Regeln übersichtlicher und modularer zu gestalten. Benutzerdefinierte Chains können von den vordefinierten Chains oder von anderen benutzerdefinierten Chains aufgerufen werden.

Um eine neue benutzerdefinierte Chain zu erstellen, verwendet man den Befehl:

```
iptables -N CUSTOM_CHAIN
```

Um eine Regel in einer benutzerdefinierten Chain zu erstellen, verwendet man den Befehl:

```
iptables -A CUSTOM_CHAIN -p tcp --dport 80 -j ACCEPT
```

Um eine Regel in einer vordefinierten Chain zu erstellen, die auf die benutzerdefinierte Chain verweist, verwendet man den Befehl:

```
iptables -A INPUT -s 192.168.1.0/24 -j CUSTOM_CHAIN
```

In diesem Beispiel leitet die Regel in der INPUT-Kette alle Pakete von der IP-Adresse 192.168.1.0/24 an die benutzerdefinierte Chain "CUSTOM_CHAIN" weiter, wo die entsprechenden Regeln in dieser benutzerdefinierten Chain angewendet werden.

Einige der wichtigsten Iptables-Kommandozeilenparameter mit Beispielen sind:

1. **-A**: Fügt eine Regel am Ende einer Kette hinzu. Beispiel: `iptables -A INPUT -p tcp --dport 22 -j ACCEPT`
2. **-I**: Fügt eine Regel am Anfang einer Kette oder an einer bestimmten Position ein. Beispiel: `iptables -I INPUT 1 -p tcp --dport 80 -j ACCEPT`
3. **-D**: Löscht eine Regel anhand ihrer Nummer oder ihrer Spezifikation. Beispiel: `iptables -D INPUT -p tcp --dport 22 -j ACCEPT`
4. **-F**: Leert (löscht alle Regeln) in einer Kette. Beispiel: `iptables -F INPUT`
5. **-L**: Listet die Regeln in einer Kette auf. Beispiel: `iptables -L`
6. **-N**: Erstellt eine neue benutzerdefinierte Kette. Beispiel: `iptables -N CUSTOM_CHAIN`
7. **-P**: Legt die Standardrichtlinie für eine Kette fest. Beispiel: `iptables -P INPUT DROP`
8. **-s**: Gibt die Quell-IP-Adresse oder das Subnetz für eine Regel an. Beispiel: `iptables -A INPUT -s 192.168.1.0/24 -j ACCEPT`

9. `-d`: Gibt die Ziel-IP-Adresse oder das Subnetz für eine Regel an. Beispiel:
`iptables -A FORWARD -d 192.168.2.0/24 -j ACCEPT`
10. `-p`: Gibt das Protokoll für eine Regel an (z.B. tcp, udp, icmp). Beispiel:
`iptables -A INPUT -p icmp -j ACCEPT`
11. `--dport`: Gibt den Zielport für eine Regel an. Beispiel: `iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
12. `-j`: Gibt die Aktion (Ziel) für eine Regel an (z.B. ACCEPT, DROP, REJECT, LOG). Beispiel: `iptables -A INPUT -p tcp --dport 22 -j DROP`

Man kann zwischen sogenannten **stateless** und **stateful** Firewalls unterscheiden:

Stateless Firewalls: Eine Stateless Firewall filtert Pakete basierend auf den Informationen in den IP-Paketheadern (z.B. Quell- und Ziel-IP-Adressen, Ports, Protokolle), ohne den Kontext oder den Status der Verbindung zu berücksichtigen. Sie wendet die Regeln auf jedes Paket einzeln an, unabhängig von früheren oder späteren Paketen in derselben Verbindung. Stateless Firewalls bieten zwar grundlegende Sicherheit, können jedoch nicht alle Angriffsarten erkennen oder abwehren, da sie den Verbindungsstatus nicht verfolgen.

Stateful Firewalls: Im Gegensatz zu Stateless Firewalls verfolgen Stateful Firewalls den Status von Netzwerkverbindungen und sind in der Lage, Pakete basierend auf dem Kontext der Verbindung zu filtern. Sie analysieren den Verlauf der Verbindung und treffen Entscheidungen auf der Grundlage von Verbindungsstatusinformationen wie SYN, ACK, FIN-Flags und Sequenznummern. Stateful Firewalls bieten eine höhere Sicherheit, da sie Angriffe besser erkennen können, die sich auf die Manipulation von Verbindungsstatusinformationen stützen, und sie ermöglichen eine differenziertere Zugriffssteuerung.

Iptables kann sowohl als Stateless als auch als Stateful Firewall fungieren, abhängig von den konfigurierten Regeln. Um Iptables als Stateful Firewall zu verwenden, kann die Verbindungsnachverfolgung (`conntrack`) eingesetzt werden. Mit dem `conntrack`-Modul können Regeln erstellt werden, die auf den Verbindungsstatus achten und somit das Filtern von Paketen basierend auf dem Verbindungsstatus ermöglichen.

Ein Beispiel, wie man Iptables konfiguriert, um als Stateful Firewall zu fungieren:

1. Aktivieren der Verbindungsnachverfolgung für etablierte und verwandte Verbindungen:

```
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED
-j ACCEPT
```

```
iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED
-j ACCEPT
```

2. Ablehnen von ungültigen Paketen:

```
iptables -A INPUT -m conntrack --ctstate INVALID -j DROP
```

Diese Regeln ermöglichen es Iptables, den Verbindungsstatus zu berücksichtigen und somit als Stateful Firewall zu agieren.

nmap

Nmap (Network Mapper) ist ein leistungsfähiges Open-Source-Tool, das zum Scannen von Netzwerken, zur Erkennung von Hosts und zur Identifizierung von offenen Ports, Betriebssystemen und Diensten verwendet wird. Nmap wird häufig von Systemadministratoren, Sicherheitsforschern und Penetrationstestern eingesetzt, um Schwachstellen in Netzwerken aufzudecken und Informationen über Netzwerkgeräte zu sammeln.

Der Funktionsumfang von Nmap umfasst:

1. Host-Entdeckung: Identifizierung von aktiven Hosts im Netzwerk.
2. Port-Scanning: Erkennung von offenen, geschlossenen oder gefilterten Ports auf Netzwerkgeräten.
3. Betriebssystem-Erkennung: Identifizierung der Betriebssysteme, die auf den erfassten Hosts laufen.
4. Diensterkennung: Identifizierung von Diensten und deren Versionen, die auf offenen Ports laufen.
5. Scriptable Engine: Verwendung von Nmap Scripting Engine (NSE) zur Automatisierung von Sicherheitsaufgaben und zum Sammeln zusätzlicher Informationen.

Ein normaler Scan und ein Stealth-Scan mit Nmap unterscheiden sich in der Art und Weise, wie sie Ports scannen und wie sie von Ziel-Hosts oder Sicherheitssystemen wie Firewalls und Intrusion Detection Systems (IDS) wahrgenommen werden. Der Hauptunterschied liegt in der Art der verwendeten Pakete und dem Grad der Unauffälligkeit der Scans.

1. Normaler Scan (TCP-Connect-Scan, **-sT**): Ein normaler Scan in Nmap ist ein TCP-Connect-Scan, bei dem das Tool versucht, eine vollständige TCP-Verbindung mit jedem Zielpport herzustellen. Nmap sendet ein SYN-Paket (Synchronize) an den Zielpport, und wenn der Port offen ist, antwortet der Ziel-Host mit einem SYN-ACK-Paket (Synchronize-Acknowledge). Nmap vervollständigt daraufhin den Verbindungsaufbau, indem es ein ACK-Paket (Acknowledge) an den Ziel-Host sendet. Schließlich wird die Verbindung mit einem FIN-Paket (Finish) geschlossen. Da bei diesem Scan eine vollständige TCP-Verbindung hergestellt wird, ist es wahrscheinlicher, dass er von Sicherheitssystemen erkannt wird.

Beispiel: `nmap -sT 192.168.1.1`

2. Stealth-Scan (SYN-Scan, **-sS**): Ein Stealth-Scan, auch als SYN-Scan oder "halb offener" Scan bezeichnet, ist unauffälliger als ein normaler Scan, da er den Verbindungsaufbau nicht abschließt. Bei einem Stealth-Scan

sendet Nmap ein SYN-Paket an den Zielport, und wenn der Port offen ist, antwortet der Ziel-Host mit einem SYN-ACK-Paket. Anstatt jedoch das ACK-Paket zum Abschluss der Verbindung zu senden, sendet Nmap ein RST-Paket (Reset) an den Ziel-Host, um den Verbindungsaufbau abubrechen. Da keine vollständige Verbindung hergestellt wird, ist dieser Scan schwieriger für Sicherheitssysteme zu erkennen.

Beispiel: `nmap -sS 192.168.1.1`

Stealth-Scans sind nützlich, um Firewalls, IDS und andere Sicherheitsmaßnahmen zu umgehen, die normalerweise Alarm schlagen, wenn sie vollständige Verbindungsanfragen erkennen. Stealth-Scans sind jedoch in der Regel langsamer als normale Scans, da sie auf Zeitüberschreitungen warten müssen, um geschlossene Ports zu erkennen. Es ist auch wichtig zu beachten, dass einige moderne Sicherheitssysteme in der Lage sind, auch Stealth-Scans zu erkennen und zu blockieren.

Hier sind einige der wichtigsten Nmap-Kommandozeilenparameter mit Beispielen:

1. **-sn**: Führt einen Ping-Scan durch, um aktive Hosts im Netzwerk zu entdecken, ohne Port-Scans durchzuführen. Beispiel: `nmap -sn 192.168.1.0/24`
2. **-p**: Gibt die zu scannenden Ports an. Sie können einen einzelnen Port, mehrere Ports oder einen Portbereich angeben. Beispiel: `nmap -p 80,443,8080 192.168.1.1` oder `nmap -p 1-1024 192.168.1.1`
3. **-sS**: Führt einen SYN-Scan durch, der ein halboffenes Scannen ermöglicht, indem nur SYN-Pakete gesendet werden, ohne die Verbindung vollständig herzustellen. Beispiel: `nmap -sS 192.168.1.1`
4. **-sU**: Führt einen UDP-Scan durch, um offene UDP-Ports zu identifizieren. Beispiel: `nmap -sU 192.168.1.1`
5. **-O**: Aktiviert die Betriebssystem-Erkennung. Beispiel: `nmap -O 192.168.1.1`
6. **-sV**: Führt eine Dienst- und Versionsprüfung auf offenen Ports durch. Beispiel: `nmap -sV 192.168.1.1`
7. **-T**: Legt die Timing-Vorlage fest, die die Geschwindigkeit und Aggressivität des Scans bestimmt (von 0 bis 5, wobei 0 am langsamsten und 5 am schnellsten ist). Beispiel: `nmap -T4 192.168.1.1`
8. **-oA**: Speichert die Scanergebnisse in verschiedenen Dateiformaten (Normal, XML, Grepable). Beispiel: `nmap -oA outputfile 192.168.1.1`
9. **-script**: Führt NSE-Skripte aus, um zusätzliche Informationen zu sammeln oder bestimmte Sicherheitsaufgaben auszuführen. Beispiel: `nmap --script http-title,http-headers 192.168.1.1`

10. `--top-ports`: Scannt eine bestimmte Anzahl der am häufigsten verwendeten Ports. Beispiel: `nmap --top-ports 10 192.168.1.1`
11. `-A`: Eine aggressive Scanoption, die mehrere Funktionen gleichzeitig ausführt, wie Betriebssystem-Erkennung, Dienst-Versionserkennung, NSE-Skriptausführung und Traceroute. Beispiel: `nmap -A 192.168.1.1`
12. `-6`: Aktiviert den IPv6-Scanmodus. Beispiel: `nmap -6 -p 80,443 [IPv6-Adresse]`